Nama	: Hasan Zubeir Pohan
NIM	: 714240045
Kelas	: 3C
Matkul	: Pemrograman 4
Tanggal Praktikum	: Rabu, 14 Oktober 2026

JURNAL PRAKTIKUM PEMROGRAMAN IV

Modul 04 – Pemrograman Berorientasi Objek Dasar

1. Tujuan Praktikum

Mahasiswa mampu memodelkan entitas pada proses logistik ke dalam kelas dan objek pada proyek SiLog. Secara khusus, praktikum ini bertujuan untuk:

- Merancang kelas **Barang**, **Gudang**, **Pengiriman**, dan **Pelanggan** beserta atribut, metode, dan relasinya dalam diagram kelas.
- Membentuk objek melalui konstruktor yang tepat, termasuk konstruktor bernama (*Barang.dariPemindaian* dan *Pengiriman.baru*) dan parameter bernama *required*.
- Menerapkan hak akses dengan awalan garis bawah serta *getter* dan *setter* sehingga atribut internal tidak dapat diubah langsung dari luar kelas (enkapsulasi).
- Menimpa metode *toString()* agar objek mudah dibaca saat dicetak.
- Menambahkan kelas Pelanggan dan atribut *pelanggan* pada kelas Pengiriman sebagai Tugas Praktikum.

2. Alat dan Bahan

No	Alat / Bahan	Keterangan
1	Flutter SDK & Dart SDK	Flutter 3.47.4 (channel stable); Dart SDK bawaan Flutter
2	Visual Studio Code	Dengan ekstensi Flutter dan Dart
3	Terminal PowerShell	Menjalankan `dart run` dan perintah Git
4	draw.io (app.diagrams.net)	Menggambar diagram kelas UML
5	Git & GitHub	https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%204
6	Proyek silog_app	Proyek yang sama dari modul sebelumnya

3. Langkah Kerja

A. Penyiapan dan Perancangan

Proyek silog_app dibuka melalui File > Open Folder pada Visual Studio Code, lalu dibuka terminal terintegrasi. Direktori `lib/model` dibuat melalui panel Explorer. Diagram kelas digambar pada draw.io (app.diagrams.net) dan disimpan sebagai gambar untuk dilampirkan pada jurnal.

B. Kelas Barang

Berkas `lib/model/barang.dart` dibuat dan diisi kelas Barang dengan konstruktor utama, konstruktor bernama `dariPemindaian`, getter dan setter `hargaSatuan`, metode `nilaiTotal`, serta `toString`.

C. Kelas Gudang

Berkas `lib/model/gudang.dart` dibuat dan diisi kelas Gudang dengan stok privat `_stok`, getter `totalStok`, `penuh`, `stok`, serta metode `terimaBarang` dan `keluarkanBarang`.

D. Kelas Pengiriman

Berkas `lib/model/pengiriman.dart` dibuat dan diisi kelas Pengiriman dengan konstruktor utama, konstruktor bernama `baru`, status privat, getter `status` dan `nilaiBarang`, serta metode `ubahStatus`.

E. Pengujian Model

Berkas `latihan/modul04_uji.dart` dibuat lalu dijalankan dengan `dart run latihan/modul04_uji.dart`. Seluruh output dicatat. Sebagai eksperimen, nilai `_stok` dicoba diubah langsung dari `main()` untuk mengamati pesan error compiler, kemudian baris tersebut dihapus kembali.

F. Tugas Praktikum

Berkas `lib/model/pelanggan.dart` dibuat dengan metode `alamatRingkas()`. Atribut `pelanggan` ditambahkan pada kelas Pengiriman beserta penyesuaian `toString()`. Diagram kelas diperbarui sehingga memuat keempat kelas.

G. Pengumpulan

Jurnal disusun sesuai sistematika dan perubahan di-*commit* dengan pesan `Pertemuan 4` pada cabang `main` dan di-*push* ke GitHub pada folder Pertemuan 4.

4. Kode Program dan Penjelasan

Setiap berkas disajikan dalam tabel yang memuat nomor baris, potongan kode, dan penjelasannya.

4.1 lib/model/barang.dart

Kelas Barang memodelkan barang yang disimpan di gudang dan dikirim kepada pelanggan.

Baris	Kode	Penjelasan

1-6	`class Barang {`
	`  final String kode;`
	`  final String nama;`
	`  final String satuan;`
	`  double _hargaSatuan;`
	`  double beratKg;`
		Deklarasi kelas dan atribut. **kode**, **nama**, **satuan** dibuat *final* karena identitas barang tidak berubah. _**hargaSatuan** diawali garis bawah sehingga privat pada berkas ini; **beratKg** tetap publik dan dapat diperbarui.

8-14	`  Barang({`
	`    required this.kode,`
	`    required this.nama,`
	`    required double hargaSatuan,`
	`    this.satuan = 'pcs',`
	`    this.beratKg = 0,`
	`  }) : _hargaSatuan = hargaSatuan;`
		Konstruktor dengan parameter bernama. Kode, nama, dan harga wajib diisi (*required*); satuan dan berat punya nilai bawaan. Atribut privat tidak dapat memakai *this._hargaSatuan* pada parameter bernama, sehingga diisi lewat *initializer list* setelah tanda titik dua.

16-24	`  /// Konstruktor bernama untuk data hasil`
	`  /// pemindaian kode barang.`
	`  Barang.dariPemindaian(String baris)`
	`      : kode = baris.split(';')[0],`
	`        nama = baris.split(';')[1],`
	`        satuan = 'pcs',`
	`        _hargaSatuan =`
	`            double.parse(baris.split(';')[2]),`
	`        beratKg = 0;`
		Konstruktor bernama untuk membentuk Barang dari teks mentah berformat *kode;nama;harga*. Teks dipecah dengan *split(';')*, lalu bagian ketiga diubah menjadi angka dengan *double.parse*.

26	`  double get hargaSatuan => _hargaSatuan;`	*Getter* untuk membaca harga satuan dari luar kelas tanpa membuka atribut privatnya.

Baris	Kode	Penjelasan

27-33	`  set hargaSatuan(double nilai) {`
	`    if (nilai < 0) {`
	`      throw ArgumentError(`
	`          'Harga satuan tidak boleh negatif');`
	`    }`
	`    _hargaSatuan = nilai;`
	`  }`
		Setter dengan validasi: nilai negatif ditolak dengan *ArgumentError*, selain itu harga diperbarui.

35-36	`  double nilaiTotal(int jumlah) =>` `      _hargaSatuan * jumlah;`	Menghitung nilai total sejumlah unit barang (harga satuan dikali jumlah).

38-42	`  @override`
	`  String toString() =>`
	`      '[$kode] $nama ($satuan) - '`
	`      'Rp${_hargaSatuan.toStringAsFixed(0)}';`
	`}`
		Menimpa *toString()* agar objek tercetak dalam bentuk yang mudah dibaca, misalnya *[B-1024] Sepatu Lari Pria (pcs) - Rp450000*.

4.2 lib/model/gudang.dart

Kelas Gudang memodelkan tempat penyimpanan barang beserta stoknya.

Baris	Kode	Penjelasan
1	`import 'barang.dart';`	Mengimpor kelas Barang karena Gudang menerima dan mengeluarkan objek Barang.

3-8	`class Gudang {`
	`  final String kode;`
	`  final String nama;`
	`  final String kota;`
	`  final int kapasitas;`
	`  final Map<String, int> _stok = {};`
		Atribut identitas gudang dan kapasitas bersifat *final*. _**stok** adalah *Map* kode barang ke jumlah, dibuat privat agar tidak dapat diubah langsung dari luar.

10-15	`  Gudang({`
	`    required this.kode,`
	`    required this.nama,`
	`    required this.kota,`
	`    this.kapasitas = 1000,`
	`  });`
		Konstruktor bernama-parameter. Kapasitas bernilai bawaan 1000 bila tidak diisi.

17-21	`  int get totalStok =>`
	`      _stok.values.fold(0, (a, b) => a + b);`
	`  bool get penuh => totalStok >= kapasitas;`
	`  Map<String, int> get stok =>`
	`      Map.unmodifiable(_stok);`
		Tiga *getter*: **totalStok** menjumlahkan seluruh stok, **penuh** bernilai true bila total mencapai kapasitas, dan **stok** mengembalikan salinan yang tidak dapat diubah (*unmodifiable*).

23-36	`  void terimaBarang(Barang barang, int jumlah) {`
	`    if (jumlah <= 0) {`
	`      throw ArgumentError(`
	`          'Jumlah penerimaan harus lebih '`
	`          'besar dari nol');`
	`    }`
	`    if (totalStok + jumlah > kapasitas) {`
	`      throw StateError(`
	`          'Kapasitas gudang $nama tidak '`
	`          'mencukupi');`
	`    }`
	`    _stok[barang.kode] =`
	`        (_stok[barang.kode] ?? 0) + jumlah;`
	`  }`
		Menambah stok. Penerimaan ditolak bila jumlah nol atau negatif (*ArgumentError*) atau melebihi kapasitas (*StateError*). Operator *??* memberi nilai awal 0 bila barang belum pernah tercatat.

38-46	`  void keluarkanBarang(Barang barang, int jumlah)`
	`{`
	`    final tersedia = _stok[barang.kode] ?? 0;`
	`    if (jumlah > tersedia) {`
	`      throw StateError(`
	`          'Stok ${barang.kode} hanya $tersedia, '`
	`          'tidak dapat mengeluarkan $jumlah');`
	`    }`
	`    _stok[barang.kode] = tersedia - jumlah;`
	`  }`
		Mengurangi stok. Bila jumlah yang diminta melebihi stok tersedia, dilempar *StateError* sehingga stok tidak pernah negatif.

Baris	Kode	Penjelasan

48-52	`  @override`
	`  String toString() =>`
	`      '$kode - $nama ($kota), '`
	`      'stok $totalStok/$kapasitas';`
	`}`
		Representasi teks gudang beserta total stok terhadap kapasitas.

4.3 lib/model/pelanggan.dart

Kelas Pelanggan (Tugas Praktikum) memodelkan penerima barang.

Baris	Kode	Penjelasan

1-5	`class Pelanggan {`
	`  final String kode;`
	`  final String nama;`
	`  final String nomorTelepon;`
	`  final String alamat;`
		Kelas baru untuk tugas praktikum. Seluruh atribut dibuat *final* karena data pelanggan diperlakukan tetap; perubahan data berarti membentuk objek baru.

7-12	`  Pelanggan({`
	`    required this.kode,`
	`    required this.nama,`
	`    required this.nomorTelepon,`
	`    required this.alamat,`
	`  });`
		Konstruktor dengan parameter bernama yang seluruhnya wajib diisi.

14-18	`  /// Mengembalikan alamat maksimal 40 karakter.`
	`  String alamatRingkas() {`
	`    if (alamat.length <= 40) return alamat;`
	`    return '${alamat.substring(0, 37)}...';`
	`  }`
		Bila alamat tidak lebih dari 40 karakter, dikembalikan apa adanya. Bila lebih, diambil 37 karakter pertama lalu ditambah tiga titik sehingga panjang hasil tetap 40 karakter.

20-23	`  @override`
	`  String toString() =>`
	`      '$kode - $nama ($nomorTelepon)';`
	`}`
		Representasi teks pelanggan.

4.4 lib/model/pengiriman.dart

Kelas Pengiriman memodelkan proses pengiriman barang dari gudang asal ke kota tujuan untuk seorang pelanggan.

Baris	Kode	Penjelasan

1-3	`import 'barang.dart';`
	`import 'gudang.dart';`
	`import 'pelanggan.dart';`
		Mengimpor ketiga kelas yang menjadi atribut Pengiriman (*pelanggan.dart* ditambahkan pada tugas praktikum).

5-13	`class Pengiriman {`
	`  final String resi;`
	`  final Gudang asal;`
	`  final String kotaTujuan;`
	`  final Barang barang;`
	`  final Pelanggan pelanggan;`
	`  final int jumlah;`
	`  final DateTime tanggalKirim;`
	`  String _status;`
		Atribut pengiriman. Seluruh data transaksi bersifat *final*; atribut **pelanggan** bertipe Pelanggan ditambahkan pada tugas praktikum. Hanya _**status** yang dapat berubah dan dibuat privat.

15-24	`  Pengiriman({`
	`    required this.resi,`
	`    required this.asal,`
	`    required this.kotaTujuan,`
	`    required this.barang,`
	`    required this.pelanggan,`
	`    required this.jumlah,`
	`    required this.tanggalKirim,`
	`    String status = 'Draft',`
	`  }) : _status = status;`
		Konstruktor utama. Status awal bernilai bawaan *Draft* dan disalin ke *_status* melalui *initializer list*.

Baris	Kode	Penjelasan

26-36	`  /// Konstruktor bernama untuk pengiriman`
	`  /// yang baru didaftarkan hari ini.`
	`  Pengiriman.baru({`
	`    required this.resi,`
	`    required this.asal,`
	`    required this.kotaTujuan,`
	`    required this.barang,`
	`    required this.pelanggan,`
	`    required this.jumlah,`
	`  }) : tanggalKirim = DateTime.now(),`
	`       _status = 'Menunggu Pickup';`
		Konstruktor bernama untuk pengiriman baru: tanggal otomatis diisi waktu saat ini dan status awal *Menunggu Pickup*.

38-40	`  String get status => _status;`
	`  double get nilaiBarang =>`
	`      barang.nilaiTotal(jumlah);`
		Getter **status** untuk membaca status privat, dan **nilaiBarang** yang menghitung nilai barang dengan memanggil *Barang.nilaiTotal*.

42-61	`  void ubahStatus(String statusBaru) {`
	`    const urutan = [`
	`      'Draft',`
	`      'Menunggu Pickup',`
	`      'Dalam Perjalanan',`
	`      'Tiba di Kota Tujuan',`
	`      'Diterima',`
	`    ];`
	`    if (!urutan.contains(statusBaru)) {`
	`      throw ArgumentError(`
	`          'Status $statusBaru tidak dikenal');`
	`    }`
	`    if (urutan.indexOf(statusBaru) <`
	`        urutan.indexOf(_status)) {`
	`      throw StateError(`
	`          'Status tidak dapat mundur dari '`
	`          '$_status ke $statusBaru');`
	`    }`
	`    _status = statusBaru;`
	`  }`
		Mengubah status sesuai urutan yang sah. Status di luar daftar ditolak (*ArgumentError*), begitu pula status yang mundur dari status saat ini (*StateError*).

63-68	`  @override`
	`  String toString() =>`
	`      '$resi | ${barang.nama} x$jumlah | '`
	`      '${asal.kota} -> $kotaTujuan | '`
	`      '${pelanggan.nama} | $_status';`
	`}`
		toString() disesuaikan: nama pelanggan ditambahkan di antara rute pengiriman dan status.

4.5 latihan/modul04_uji.dart

Berkas pengujian yang membentuk objek, mengubah stok, dan menguji penolakan operasi yang tidak sah.

Baris	Kode	Penjelasan

1-4	`import '../lib/model/barang.dart';`
	`import '../lib/model/gudang.dart';`
	`import '../lib/model/pengiriman.dart';`
	`import '../lib/model/pelanggan.dart';`
		Mengimpor keempat berkas model dari direktori *lib/model*.

6-14	`void main() {`
	`  final sepatu = Barang(`
	`    kode: 'B-1024',`
	`    nama: 'Sepatu Lari Pria',`
	`    hargaSatuan: 450000,`
	`    beratKg: 0.9,`
	`  );`
	`  final kaos = Barang.dariPemindaian(`
	`      'B-2048;Kaos Katun;85000');`
		Membentuk dua objek Barang: **sepatu** lewat konstruktor utama dan **kaos** lewat konstruktor bernama *dariPemindaian*.

Baris	Kode	Penjelasan

15-28	`  final gudangBandung = Gudang(`
	`    kode: 'GD-BDG',`
	`    nama: 'Gudang Utama Bandung',`
	`    kota: 'Bandung',`
	`    kapasitas: 500,`
	`  );`
	`  final budi = Pelanggan(`
	`    kode: 'P-001',`
	`    nama: 'Budi Santoso',`
	`    nomorTelepon: '081234567890',`
	`    alamat: 'Jl. Soekarno Hatta No. 123, '`
	`        'Kelurahan Cibiru, Kota Bandung, '`
	`        'Jawa Barat',`
	`  );`
		Membentuk objek Gudang berkapasitas 500 dan objek Pelanggan dengan alamat yang sengaja dibuat lebih dari 40 karakter untuk menguji *alamatRingkas()*.

30-34	`  gudangBandung.terimaBarang(sepatu, 120);`
	`  gudangBandung.terimaBarang(kaos, 300);`
	`  print(gudangBandung);`
	`  print('Rincian stok: '`
	`      '${gudangBandung.stok}');`
		Gudang menerima 120 sepatu dan 300 kaos (total 420 dari kapasitas 500), lalu mencetak ringkasan gudang dan rincian stok.

36-48	`  final kirim = Pengiriman.baru(`
	`    resi: 'SLG-2026-0007',`
	`    asal: gudangBandung,`
	`    kotaTujuan: 'Surabaya',`
	`    barang: sepatu,`
	`    pelanggan: budi,`
	`    jumlah: 10,`
	`  );`
	`  gudangBandung.keluarkanBarang(`
	`      sepatu, kirim.jumlah);`
	`  print(kirim);`
	`  print('Stok setelah pengiriman: '`
	`      '${gudangBandung.stok}');`
		Membuat pengiriman baru untuk 10 sepatu, mengeluarkan 10 sepatu dari gudang, lalu mencetak pengiriman dan stok terbaru.

50-53	`  print('Nilai barang: Rp'`
	`      '${kirim.nilaiBarang.toStringAsFixed(0)}');`
	`  print('Alamat ringkas: '`
	`      '${budi.alamatRingkas()}');`
		Mencetak nilai barang yang dikirim dan alamat ringkas pelanggan.

55-56	`  kirim.ubahStatus('Dalam Perjalanan');` `  print('Status terbaru: ${kirim.status}');`	Mengubah status ke langkah berikutnya yang sah, lalu mencetak status baru.

58-62	`  try {`
	`    kirim.ubahStatus('Draft');`
	`  } on StateError catch (e) {`
	`    print('Ditolak: ${e.message}');`
	`  }`
		Uji operasi tidak sah 1: status mundur ke *Draft* harus ditolak dengan *StateError*.

64-69	`  try {`
	`    gudangBandung.keluarkanBarang(`
	`        sepatu, 9999);`
	`  } on StateError catch (e) {`
	`    print('Ditolak: ${e.message}');`
	`  }`
		Uji operasi tidak sah 2: mengeluarkan 9999 sepatu melebihi stok harus ditolak.

71-76	`  try {`
	`    sepatu.hargaSatuan = -1;`
	`  } on ArgumentError catch (e) {`
	`    print('Ditolak: ${e.message}');`
	`  }`
	`}`
		Uji operasi tidak sah 3: setter harga menolak nilai negatif dengan *ArgumentError*.

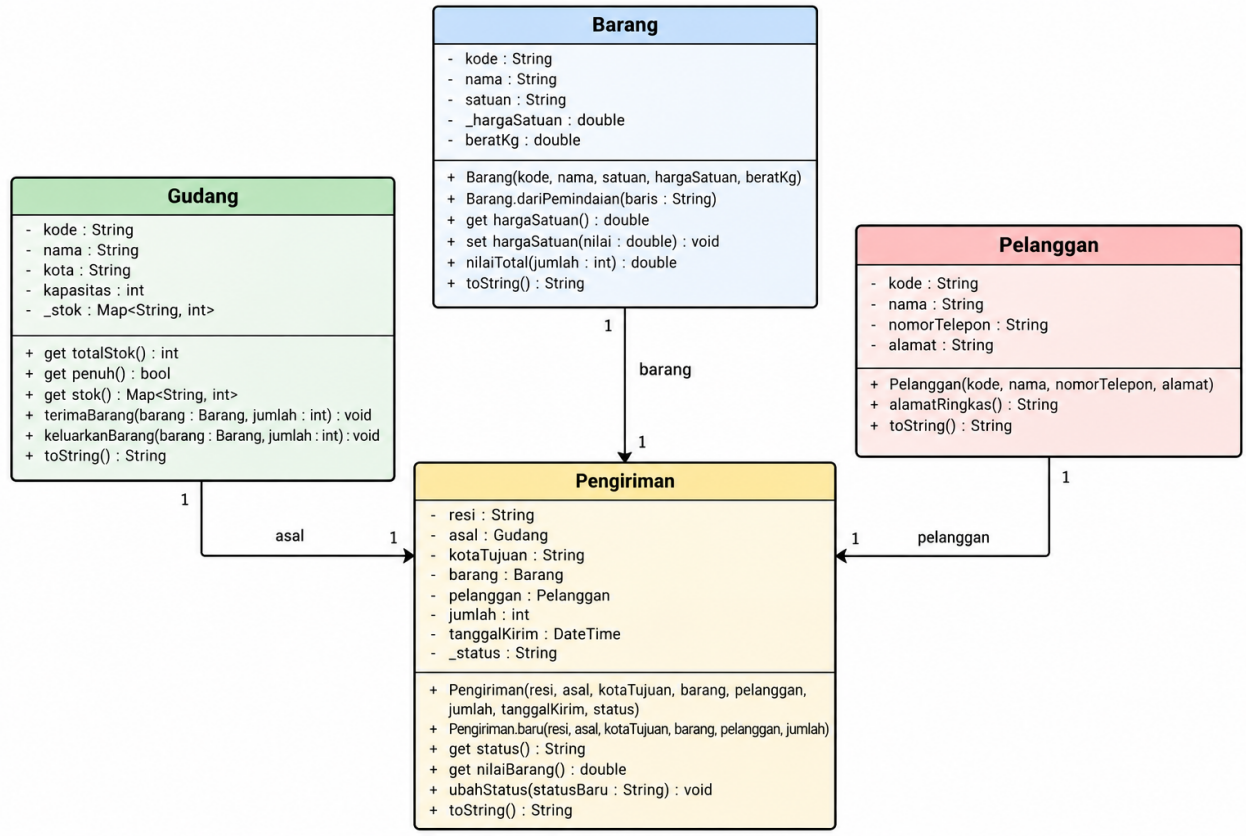
4.6 Konsep yang Diterapkan

- **Kelas dan objek.** Barang, Gudang, Pengiriman, dan Pelanggan adalah cetak biru; objek seperti *sepatu* (B-1024) atau *gudangBandung* adalah wujud nyatanya saat program berjalan.
- **Konstruktor.** Parameter bernama dengan *required* memaksa data wajib diisi, sedangkan konstruktor bernama memberi cara pembentukan objek yang berbeda: *Barang.dariPemindaian* dari teks mentah dan *Pengiriman.baru* dengan tanggal dan status awal otomatis.
- **Hak akses.** Dart tidak memiliki kata kunci private atau public; atribut yang diawali garis bawah (*_stok*, *_hargaSatuan*, *_status*) hanya dapat diakses dari berkas yang sama.

- **Getter dan setter.** Akses ke atribut privat dibuka secara terkendali, dan perubahan nilai disertai pemeriksaan kewajaran.

5. Hasil Eksekusi dan Analisis

5.1 Diagram Kelas



Gambar 1. Diagram kelas Barang, Gudang, Pengiriman, dan Pelanggan.

Diagram memuat empat kelas. Kelas Pengiriman berada di pusat dan memiliki tiga relasi asosiasi: ke **Gudang** dengan peran *asal*, ke **Barang** dengan peran *barang*, dan ke **Pelanggan** dengan peran *pelanggan*. Pada diagram, tiap relasi bermultiplisitas 1 pada kedua sisi. Pada implementasi, atribut tersebut hanya menyimpan referensi objek, sehingga satu objek Gudang, Barang, atau Pelanggan yang sama dapat dirujuk oleh beberapa Pengiriman.

Relasi	Peran	Keterangan
Gudang → Pengiriman	asal	Setiap pengiriman berangkat dari satu gudang asal.
Barang → Pengiriman	barang	Setiap pengiriman membawa satu jenis barang dengan jumlah tertentu.
Pelanggan → Pengiriman	pelanggan	Setiap pengiriman ditujukan kepada satu pelanggan.

5.2 Output modul04_uji.dart

Program dijalankan dengan perintah `dart run latihan/modul04_uji.dart`. Output yang dihasilkan dicatat pada tabel berikut, berurutan sesuai eksekusi kode.

No	Perintah pada kode	Output	Keterangan
1	`print(gudangBandung)`	`GD-BDG - Gudang Utama Bandung (Bandung), stok 420/500`	Setelah menerima 120 sepatu dan 300 kaos
2	`print('Rincian stok: ...')`	`Rincian stok: {B-1024: 120, B-2048: 300}`	Rincian stok per kode barang

No	Perintah pada kode	Output	Keterangan
3	`print(kirim)`	`SLG-2026-0007 | Sepatu Lari Pria x10 | Bandung -> Surabaya | Budi Santoso | Menunggu Pickup`	Pengiriman baru; nama pelanggan tampil pada toString()
4	`print('Stok setelah pengiriman: ...')`	`Stok setelah pengiriman: {B-1024: 110, B-2048: 300}`	Stok sepatu berkurang 10
5	`print('Nilai barang: ...')`	`Nilai barang: Rp4500000`	450000 x 10
6	`print('Alamat ringkas: ...')`	`Alamat ringkas: Jl. Soekarno Hatta No. 123, Kelurahan...`	Alamat dipotong menjadi 40 karakter
7	`print('Status terbaru: ...')`	`Status terbaru: Dalam Perjalanan`	Perubahan status yang sah
8	`kirim.ubahStatus('Draft')`	`Ditolak: Status tidak dapat mundur dari Dalam Perjalanan ke Draft`	StateError: status tidak boleh mundur
9	`keluarkanBarang(sepatu, 9999)`	`Ditolak: Stok B-1024 hanya 110, tidak dapat mengeluarkan 9999`	StateError: melebihi stok
10	`sepatu.hargaSatuan = -1`	`Ditolak: Harga satuan tidak boleh negatif`	ArgumentError dari setter

Output menunjukkan tiga hal yang diminta modul: **pembentukan objek** (baris 1 dan 3, melalui konstruktor utama dan konstruktor bernama), **perubahan stok** (baris 2 dan 4: stok sepatu turun dari 120 menjadi 110 setelah pengiriman 10 unit), dan **penolakan operasi yang tidak sah** (baris 8 sampai 10). Setiap penolakan dilempar sebagai exception dan ditangkap pada blok *try/on*, sehingga program tidak berhenti mendadak.

5.3 Pengujian Enkapsulasi (_stok)

Sebagai eksperimen sementara, baris berikut ditambahkan pada fungsi `main()` di `modul04_uji.dart`:

Baris yang ditambahkan	Hasil
`gudangBandung._stok['B-1024'] = -5;`	Error pada compiler dan analyzer: `The getter '_stok' isn't defined for the class 'Gudang'.` Program tidak dapat dijalankan.

Pengujian ini merupakan eksperimen sementara dan bukan kondisi akhir kode; baris tersebut dihapus setelah pengamatan. Error muncul karena pengenal berawalan garis bawah pada Dart bersifat privat terhadap *library* (berkas). *_stok* dideklarasikan di *gudang.dart*, sedangkan *main()* berada di *modul04_uji.dart*, sehingga dari sudut pandang berkas pengujian anggota tersebut tidak ada. Inilah penerapan **enkapsulasi**: keadaan internal gudang disembunyikan, dan stok hanya dapat berubah melalui *terimaBarang* dan *keluarkanBarang* yang memeriksa kewajaran nilai. Getter *stok* pun mengembalikan *Map.unmodifiable*, sehingga salinan yang dibaca dari luar juga tidak dapat dipakai untuk mengubah stok asli.

6. Jawaban Tugas Praktikum

6.1 Kelas Pelanggan dan alamatRingkas()

Kelas Pelanggan ditambahkan pada `lib/model/pelanggan.dart` dengan atribut kode, nama, nomorTelepon, dan alamat (kode lengkap pada bagian 4.3). Metode `alamatRingkas()` mengembalikan alamat apa adanya bila panjangnya tidak lebih dari 40 karakter; bila lebih, diambil 37 karakter pertama ditambah tiga titik sehingga hasilnya tepat 40 karakter. Pada pengujian, alamat *Jl. Soekarno Hatta No. 123, Kelurahan Cibiru, Kota Bandung, Jawa Barat* menghasilkan *Jl. Soekarno Hatta No. 123, Kelurahan...* (baris 6 pada tabel output).

6.2 Atribut pelanggan pada Pengiriman

Atribut `final Pelanggan pelanggan` ditambahkan pada kelas Pengiriman dan menjadi parameter *required* pada konstruktor utama maupun konstruktor bernama *Pengiriman.baru*. Metode `toString()` disesuaikan sehingga memuat nama pelanggan:

Versi	Format toString()
Sebelum	`$resi | ${barang.nama} x$jumlah | ${asal.kota} -> $kotaTujuan | $_status`
Sesudah	`$resi | ${barang.nama} x$jumlah | ${asal.kota} -> $kotaTujuan | ${pelanggan.nama} | $_status`

6.3 Diagram kelas akhir dan alasan pemilihan atribut

Diagram kelas akhir ditampilkan pada Gambar 1 (bagian 5.1). Pada diagram, seluruh atribut diberi tanda minus (tidak terbuka bebas). Pada implementasi Dart, atribut yang benar-benar privat adalah yang berawalan garis bawah, sedangkan atribut lain dibuat *final* sehingga dapat dibaca tetapi tidak dapat diubah setelah objek terbentuk. Pembagian dan alasannya sebagai berikut:

Kelas	Atribut	Jenis	Alasan
Barang	kode, nama, satuan	final	Identitas barang tidak berubah selama objek hidup; kode yang berubah berarti barang yang berbeda.
	_hargaSatuan	privat	Harga harus lolos validasi (tidak boleh negatif), sehingga hanya diubah lewat setter yang memeriksa nilai.
	beratKg	publik, non-final	Berat dapat diperbarui (misalnya setelah penimbangan ulang) dan tidak ada aturan bisnis yang perlu dijaga.
Gudang	kode, nama, kota, kapasitas	final	Identitas dan daya tampung gudang bersifat tetap.
	_stok	privat	Stok tidak boleh negatif atau melebihi kapasitas; perubahan hanya lewat terimaBarang dan keluarkanBarang. Getter stok mengembalikan salinan tak dapat diubah.
Pengiriman	resi, asal, kotaTujuan, barang, pelanggan, jumlah, tanggalKirim	final	Merupakan catatan transaksi yang tidak boleh diubah setelah dibuat.
	_status	privat	Status satu-satunya data yang berubah dan harus mengikuti urutan sah, sehingga hanya diubah lewat ubahStatus.
Pelanggan	kode, nama, nomorTelepon, alamat	final	Data pelanggan diperlakukan tetap pada model ini; perubahan data dilakukan dengan membentuk objek baru.

6.4 Pemetaan kelas ke entitas nyata

Entitas nyata	Kelas	Atribut utama	Perilaku utama
Barang di gudang	Barang	kode, nama, satuan, harga satuan, berat	Menghitung nilai total sejumlah unit
Gudang penyimpanan	Gudang	kode, nama, kota, kapasitas, stok	Menerima dan mengeluarkan barang
Pengiriman barang	Pengiriman	resi, kota tujuan, jumlah, tanggal, status, asal, barang, pelanggan	Mengubah status sesuai urutan yang sah
Pelanggan	Pelanggan	kode, nama, nomor telepon, alamat	Menyajikan alamat ringkas

6.5 Tabel skenario uji

No	Skenario	Input	Output yang diharapkan	Output yang diperoleh	Status
1	Konstruktor utama	Barang(kode: B-1024, ...)	Objek terbentuk	sepatu terbentuk dan dapat dipakai pada pengiriman	Sesuai
2	Konstruktor bernama Barang	Barang.dariPemindaian('B-2048;Kaos Katun;85000')	Objek terbentuk dari teks mentah	kaos terbentuk; tersimpan di stok dengan kode B-2048	Sesuai
3	Terima barang	terimaBarang(sepatu, 120) dan (kaos, 300)	Total stok 420/500	GD-BDG - Gudang Utama Bandung (Bandung), stok 420/500	Sesuai
4	Konstruktor bernama Pengiriman	Pengiriman.baru(...)	Status awal Menunggu Pickup	Baris pengiriman berakhir dengan Menunggu Pickup	Sesuai
5	Keluarkan barang	keluarkanBarang(sepatu, 10)	Stok sepatu 110	{B-1024: 110, B-2048: 300}	Sesuai
6	Nilai barang	nilaiBarang untuk 10 sepatu	Rp4500000	Nilai barang: Rp4500000	Sesuai
7	Alamat ringkas	Alamat 70 karakter lebih	Maksimal 40 karakter	Jl. Soekarno Hatta No. 123, Kelurahan...	Sesuai
8	Ubah status maju	ubahStatus('Dalam Perjalanan')	Status berubah	Status terbaru: Dalam Perjalanan	Sesuai
9	Ubah status mundur	ubahStatus('Draft')	StateError, status ditolak	Ditolak: Status tidak dapat mundur dari Dalam Perjalanan ke Draft	Sesuai
10	Stok melebihi tersedia	keluarkanBarang(sepatu, 9999)	StateError, ditolak	Ditolak: Stok B-1024 hanya 110, tidak dapat mengeluarkan 9999	Sesuai
11	Harga negatif	sepatu.hargaSatuan = -1	ArgumentError, ditolak	Ditolak: Harga satuan tidak boleh negatif	Sesuai
12	Ubah _stok dari luar kelas	gudangBandung._stok[...] = -5	Error compiler	The getter '_stok' isn't defined for the class 'Gudang'.	Sesuai

7. Kesimpulan dan Kendala

Kesimpulan. Praktikum Modul 04 berhasil dilaksanakan. Entitas proses logistik berhasil dipetakan menjadi kelas Barang, Gudang, Pengiriman, dan Pelanggan dengan relasi yang digambarkan pada diagram kelas. Objek dibentuk melalui konstruktor utama dan konstruktor bernama, dan atribut internal (*_hargaSatuan*, *_stok*, *_status*) tidak dapat diubah langsung dari luar kelas, terbukti dari error compiler ketika *_stok* diakses dari *main()*. Perubahan stok dan status hanya terjadi melalui metode yang memeriksa kewajaran nilai, dan seluruh operasi tidak sah (status mundur, stok berlebih, harga negatif) ditolak dengan exception yang sesuai. Tugas praktikum berupa kelas Pelanggan dan atribut *pelanggan* pada Pengiriman juga telah diterapkan.

Kendala. Pertama, parameter inisialisasi *this._hargaSatuan* tidak dapat dipakai pada parameter bernama, sehingga nilainya diisi melalui *initializer list* setelah tanda titik dua pada konstruktor. Kedua, hak akses Dart ditentukan per berkas (*library*), bukan per kelas; karena itu pengujian enkapsulasi harus dilakukan dari berkas yang berbeda dengan tempat kelas dideklarasikan. Ketiga, penambahan atribut *pelanggan* pada Pengiriman mengharuskan seluruh pemanggil konstruktor (termasuk *modul04_uji.dart*) menyertakan argumen tersebut, dan diagram kelas harus diperbarui agar konsisten dengan kode.

8. Tautan Repositori dan Riwayat Commit

Repositori GitHub: https://github.com/Nogi3201/silog-714240045

Cabang: https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%204

Riwayat commit: Pertemuan 4 (commit d6a8467, cabang main)